What is Scaling?

Feature Engineering-এর আগের পর্বে আমরা শিখেছিলাম কীভাবে missing values handle করতে হয়, outlier detect করতে হয়, এবং categorical data encode করতে হয়। এখন আমাদের dataset-এ সব values numeric — কিন্তু একটা নতুন সমস্যা আছে। বিভিন্ন feature-এর values বিভিন্ন range-এ থাকে। এই সমস্যাটা solve করার জন্যই আমরা **scaling** শিখবো।

Scaling কেন দরকার?

একটা real example দিয়ে শুরু করি। Titanic dataset-এ দেখো বিভিন্ন feature-এর range কতটা আলাদা:

```
Feature   |  Min  |  Max
----------|-------|------
Age       |  0    |  80
Fare      |  0    |  512
SibSp     |  0    |  8
Parch     |  0    |  6
```

`Fare` এর range অনেক বড় — 0 থেকে 512 পর্যন্ত। `SibSp` মাত্র 0 থেকে 8। এই দুটো feature যখন একসাথে কোনো machine learning model-এ যাবে, তখন model মনে করবে `Fare` অনেক বেশি গুরুত্বপূর্ণ — শুধুমাত্র তার সংখ্যা বড় বলে। কিন্তু বাস্তবে সেটা সত্যি নাও হতে পারে।

এই সমস্যাটা বিশেষভাবে দেখা যায় **distance-based algorithms**-এ — যেমন KNN, SVM, এবং Linear Regression। এই algorithms গুলো feature-এর actual numeric value দেখে সিদ্ধান্ত নেয়। তাই বড় range-এর feature ছোট range-এর feature-কে dominate করে ফেলে।

Scaling কী করে?

Scaling হলো এমন একটা process যেখানে আমরা প্রতিটা feature-এর values-কে একটা **common range**-এ নিয়ে আসি। এতে model সব feature-কে সমান সুযোগ দিয়ে শিখতে পারে।

Scaling-এর তিনটা প্রধান পদ্ধতি আছে — এই module-এ আমরা এই তিনটাই শিখবো:

```
Method            |  What it does              |  Best for
------------------|----------------------------|---------------------------
Standard Scaling  |  Mean=0, Std=1             |  Normal distribution
MinMax Scaling    |  Transforms to 0-1 range   |  When bounded range needed
Robust Scaling    |  Uses Median and IQR       |  When outliers are present
```

একটা Intuition

ধরো তুমি তিনজন ছাত্রের performance compare করছো। একজনের মার্কস 0-100 স্কেলে, আরেকজনের GPA 0-4 স্কেলে, আরেকজনের attendance percentage। এই তিনটা directly compare করা যাবে না। কিন্তু যদি সবকিছু একই scale-এ নিয়ে আসো, তাহলে fair comparison সম্ভব। Scaling ঠিক এই কাজটাই করে — machine learning model-এর জন্য।

Scaling কখন করতে হয়?

একটা গুরুত্বপূর্ণ নিয়ম মনে রাখতে হবে — scaler সবসময় **training data-তে fit** করতে হয়, এবং সেই fit করা scaler দিয়ে **test data-কে transform** করতে হয়। কখনো test data-তে আলাদা করে fit করবে না। কারণ real world-এ test data আগে থেকে দেখা থাকে না।

```
# সঠিক পদ্ধতি
scaler.fit(X_train)                    # শুধু train data দেখে শেখে
X_train = scaler.transform(X_train)    # train transform করো
X_test = scaler.transform(X_test)      # same scaler দিয়ে test transform করো
```

Summary

Scaling হলো feature engineering-এর শেষ এবং গুরুত্বপূর্ণ একটা ধাপ। বিভিন্ন feature-এর বিভিন্ন range থাকলে machine learning model ঠিকমতো শিখতে পারে না। Scaling সব feature-কে একটা common range-এ এনে এই সমস্যা সমাধান করে। পরের পর্বে আমরা প্রথম পদ্ধতি **Standard Scaling** বিস্তারিত শিখবো।

Standard Scaling

আগের পর্বে আমরা বুঝেছি কেন scaling দরকার — বিভিন্ন feature-এর বিভিন্ন range model-এর শেখার প্রক্রিয়াকে bias করে ফেলে। এই সমস্যা সমাধানের প্রথম এবং সবচেয়ে জনপ্রিয় পদ্ধতি হলো **Standard Scaling**, যাকে **Standardization**-ও বলা হয়।

Standard Scaling কীভাবে কাজ করে?

Standard Scaling প্রতিটা feature-এর value থেকে সেই feature-এর **mean** বাদ দেয়, তারপর **standard deviation** দিয়ে ভাগ করে।

Formula টা এরকম:

```
z = (x - mean) / std

যেখানে:
  x    = original value
```

```
mean = feature-এর সব value-এর গড়
std  = feature-এর standard deviation
z    = scaled value (z-score নামেও পরিচিত)
```

এই scaled value-কে **z-score** বলা হয়। z-score বলে দেয় — একটা value তার mean থেকে কতটা **standard deviation** দূরে আছে।

Intuition — z-score মানে আসলে কী?

ধরো তুমি একটা ক্লাসে পরীক্ষা দিলে। তোমার মার্কস 75। এখন প্রশ্ন হলো — এই 75 কি ভালো নাকি খারাপ?
শুধু 75 দেখে বলা সম্ভব না। কিন্তু যদি জানো ক্লাসের mean 60 এবং std 10, তাহলে z = (75 - 60) / 10 = 1.5। এর মানে তোমার মার্কস mean-এর চেয়ে **1.5 standard deviation** উপরে — তুমি ক্লাসে ভালো করেছো। এই z-score দিয়ে যেকোনো পরীক্ষার result compare করা যায়, scale যাই হোক।
Standard Scaling ঠিক এই কাজটাই করে প্রতিটা feature-এর জন্য। Fare হোক বা Age — সব feature-এর value z-score-এ convert হয়ে যায়। তখন সব feature একই ভাষায় কথা বলে।

Step-by-step Example

Titanic dataset-এর `Age` column-এ mean = 29.64 এবং std = 13.08। প্রতিটা value থেকে mean বাদ দিয়ে std দিয়ে ভাগ করলে পাই:

```
Original Age  |  Scaled Value
--------------|---------------
22            |  -0.58
29            |  -0.05
38            |   0.64
60            |   2.32
```

22 বছর বয়সী যাত্রী mean-এর চেয়ে ছোট, তাই তার z-score negative। 60 বছর বয়সী যাত্রী mean-এর অনেক উপরে, তাই z-score 2.32। Scaling-এর পরে verify করলে দেখবো mean = 0.0 এবং std = 1.0।

Distribution কি বদলায়?

Scaling-এর পরে **data-র shape** বা **distribution** একদম বদলায় না — শুধু scale বদলায়।

```
Before Scaling:  mean = 29.64,  std = 13.08,  range: 0 – 80
After Scaling:   mean = 0.0,    std = 1.0,    range: -2.27 – 3.84
```

Data-র যে values আগে কাছাকাছি ছিল, scaling-এর পরেও কাছাকাছি থাকবে। একটা rubber band-এর মতো ভাবো — টানলে আকার লম্বা হয়, কিন্তু rubber band-এর উপরের দাগগুলোর relative position বদলায় না।

Outlier কেন সমস্যা করে?

Standard Scaling mean এবং std ব্যবহার করে — এবং এই দুটোই outlier-এর প্রতি **sensitive**। ধরো Age column-এ একটা outlier আছে:

```
Without outlier:  mean = 30,   std = 5.5


With outlier:     mean = 51,   std = 58.3
```

Outlier আসার পরে mean এবং std দুটোই অনেক বদলে গেছে। এখন এই distorted mean এবং std দিয়ে scale করলে normal values গুলো সব 0-এর কাছে জমে যাবে — model তখন এই values-এর মধ্যে পার্থক্য ঠিকমতো বুঝতে পারবে না। এই কারণেই outlier থাকলে Standard Scaling ব্যবহার করা উচিত না।

কোন Algorithms-এ Standard Scaling দরকার?

সব algorithm-এ scaling দরকার নেই। Decision Tree বা Random Forest-এর মতো tree-based algorithms feature-এর scale দেখে না। কিন্তু কিছু algorithm আছে যেখানে scaling অবশ্যই দরকার:

```
Algorithm          |  Why scaling is needed
-------------------|----------------------------------------
Linear Regression  |  Gradient descent converge করতে
Logistic Reg.      |  Same reason
KNN                |  Distance calculation scale-dependent
SVM                |  Margin calculation scale-dependent
PCA                |  Variance calculation scale-dependent
```

Code

```
from sklearn.preprocessing import StandardScaler

ss = StandardScaler()

# শুধু train data দিয়ে fit করো — mean এবং std শেখে
ss.fit(X_train[['Age']])

# train এবং test দুটোই transform করো
X_train['Age'] = ss.transform(X_train[['Age']]).ravel()
X_test['Age'] = ss.transform(X_test[['Age']]).ravel()
```

```

# verify করো
round(X_train['Age'].mean(), 5)  # output: 0.0
round(X_train['Age'].std(), 5)   # output: 1.0
```

`fit()` করার সময় scaler training data-র mean এবং std মনে রেখে দেয়। `transform()` করার সময় সেই stored mean এবং std দিয়ে প্রতিটা value-কে scale করে। Test data-তে আলাদা করে `fit()` করা হয় না — কারণ real world-এ নতুন data আসলে তাকে training distribution অনুযায়ী scale করতে হবে।

Summary

Standard Scaling প্রতিটা value থেকে mean বাদ দিয়ে std দিয়ে ভাগ করে — এই scaled value-কে z-score বলে। Scaling-এর পরে data-র mean হয় 0 এবং std হয় 1, কিন্তু distribution-এর shape বদলায় না। এটা normally distributed data-র জন্য সবচেয়ে উপযুক্ত, কিন্তু outlier থাকলে কার্যকর নয়। পরের পর্বে আমরা **MinMax Scaling** শিখবো — যেটা data-কে একটা fixed range-এ নিয়ে আসে।

MinMax Scaling

আগের পর্বে আমরা Standard Scaling শিখেছি — যেটা data-কে mean 0 এবং std 1-এ নিয়ে আসে। কিন্তু কখনো কখনো আমাদের দরকার হয় data-কে একটা **fixed range**-এ নিয়ে আসতে — যেমন 0 থেকে 1-এর মধ্যে। এই কাজটা করে **MinMax Scaling**।

MinMax Scaling কীভাবে কাজ করে?

MinMax Scaling প্রতিটা value থেকে সেই feature-এর **minimum** বাদ দেয়, তারপর feature-এর **range** দিয়ে ভাগ করে। Range মানে maximum থেকে minimum বাদ দেওয়া।

Formula টা এরকম:

```
x_scaled = (x - min) / (max - min)

যেখানে:
  x        = original value
  min      = feature-এর সবচেয়ে ছোট value
  max      = feature-এর সবচেয়ে বড় value
  x_scaled = scaled value, সবসময় 0 থেকে 1-এর মধ্যে
```

Intuition

ধরো তোমার কাছে একটা ruler আছে যেটা 10cm থেকে 50cm পর্যন্ত। তুমি এটাকে 0 থেকে 1-এ convert করতে চাও। 10cm হবে 0, 50cm হবে 1, এবং মাঝের সব values সেই অনুযায়ী proportionally map হবে। MinMax Scaling ঠিক এই কাজটাই করে।

এই formula-র একটা সুন্দর property আছে — **minimum value** সবসময় **0** হয়, **maximum value** সবসময় **1** হয়। বাকি সব values এই দুটোর মাঝে proportionally বসে যায়।

Step-by-step Example

Titanic dataset-এর `Fare` column-এ min = 0 এবং max = 512। কিছু values দেখি:

```
Original Fare  |  Scaled Value
---------------|---------------
0              |  0.00
128            |  0.25
256            |  0.50
512            |  1.00
```

0 Fare সবচেয়ে ছোট — scaled value 0। 512 সবচেয়ে বড় — scaled value 1। 256 ঠিক মাঝখানে — scaled value 0.50। সব values 0 থেকে 1-এর মধ্যে আসে।

Standard Scaling-এর সাথে পার্থক্য

```
Property          |  Standard Scaling  |  MinMax Scaling
------------------|--------------------|-----------------
Output range      |  Not fixed         |  Always 0 to 1
Mean after scale  |  Always 0          |  Not fixed
Outlier effect    |  High              |  Very high
Distribution      |  Shape preserved   |  Shape preserved
```

Standard Scaling-এ output range fixed থাকে না — data কতটা spread তার উপর নির্ভর করে। MinMax Scaling-এ output সবসময় 0 থেকে 1। কিন্তু MinMax Scaling outlier-এর প্রতি আরো বেশি **sensitive** — কারণ এটা min এবং max ব্যবহার করে, আর একটা outlier সরাসরি min বা max হয়ে যেতে পারে।

ধরো Fare column-এ একটা outlier আছে — কেউ 5000 টাকা fare দিয়েছে। তাহলে max হবে 5000, এবং বাকি সব normal values 0 থেকে 0.1-এর মধ্যে জমে যাবে। Model তখন তাদের মধ্যে পার্থক্য বুঝতে পারবে না।

কখন MinMax Scaling ব্যবহার করবে?

MinMax Scaling সবচেয়ে ভালো কাজ করে যখন data-তে **outlier** নেই এবং একটা **bounded range** দরকার। Neural network-এ input সাধারণত 0 থেকে 1-এর মধ্যে রাখা হয় — সেক্ষেত্রে MinMax Scaling উপযুক্ত। Titanic dataset-এ `Fare` column-এ আগেই outlier clipping করা হয়েছিল, তাই সেখানে MinMax Scaling ব্যবহার করা হয়েছে।

Code

```
from sklearn.preprocessing import MinMaxScaler

mms = MinMaxScaler()

# শুধু train data দিয়ে fit করো — min এবং max শেখে
mms.fit(X_train[['Fare']])

# train এবং test দুটোই transform করো
X_train['Fare'] = mms.transform(X_train[['Fare']]).ravel()
X_test['Fare'] = mms.transform(X_test[['Fare']]).ravel()

# verify করো
X_train['Fare'].describe()
# min = 0.0, max = 1.0
```

`fit()` করার সময় scaler training data-র min এবং max মনে রেখে দেয়। Test data-তে যদি কোনো value training-এর max-এর চেয়ে বড় হয়, তাহলে সেটা 1-এর বেশি হয়ে যাবে — এটা MinMax Scaling-এর একটা সীমাবদ্ধতা।

Summary

MinMax Scaling প্রতিটা value-কে 0 থেকে 1-এর range-এ নিয়ে আসে। এটা outlier-এর প্রতি অনেক sensitive — তাই ব্যবহারের আগে অবশ্যই outlier handle করতে হবে। Data-তে outlier না থাকলে এবং bounded range দরকার হলে এটা আদর্শ choice। পরের পর্বে আমরা **Robust Scaling** শিখবো — যেটা outlier থাকলেও ভালো কাজ করে।

Robust Scaling

আগের দুটো পর্বে আমরা Standard Scaling এবং MinMax Scaling শিখেছি। দুটোরই একটা common সমস্যা আছে — **outlier** থাকলে দুটোই ভালো কাজ করে না। Standard Scaling mean এবং std ব্যবহার করে যেগুলো outlier-এ distort হয়, MinMax Scaling min এবং max ব্যবহার করে যেগুলো outlier হয়ে যেতে পারে। এই সমস্যা সমাধান করতেই এসেছে **Robust Scaling**।

Robust Scaling কীভাবে কাজ করে?

Robust Scaling mean বা min-max ব্যবহার করে না। এটা ব্যবহার করে **median** এবং **IQR (Interquartile Range)**। Median এবং IQR দুটোই outlier-এর প্রতি **robust** — অর্থাৎ outlier থাকলেও এরা বেশি বদলায় না। Formula টা এরকম:

```
x_scaled = (x - median) / IQR

যেখানে:
  x       = original value
  median  = feature-এর মধ্যবর্তী value (50th percentile)
  IQR     = Q3 - Q1 (75th percentile - 25th percentile)
```

Intuition — কেন Median এবং IQR?

Mean একটা outlier-এ অনেক বদলে যায়। কিন্তু median বদলায় না — কারণ median শুধু মাঝের value দেখে, extreme values দেখে না। একটা example দিয়ে বুঝি:

```
Data:              [10, 12, 13, 14, 15, 16, 200]

Mean:              40.0   (outlier 200 mean-কে টেনে তুলেছে)
Median:            14.0   (outlier থাকলেও মাঝের value একই)

Std:               68.4   (outlier std-কে বিশাল করেছে)
IQR (Q3 - Q1):     3.5    (outlier IQR-কে প্রায় বদলায়নি)
```

Median এবং IQR outlier-এর উপস্থিতিতেও stable থাকে। তাই Robust Scaling outlier থাকলেও বাকি data-কে সঠিকভাবে scale করতে পারে।

Step-by-step Example

`marks_dataset.csv`-এর `maths_marks` column দিয়ে দেখি। ধরো কিছু values এবং তাদের stats:

```
Stats of maths_marks:
  median (Q2)  =  52.5
```

```
Q1           =  27.0
Q3           =  70.25
IQR          =  43.25
```

এখন কিছু values scale করি:

```
Original Mark  |  Scaled Value
---------------|---------------
27.0           |  -0.59
52.5           |   0.00
70.25          |   0.41
100            |   1.10
```

Median value 52.5 scaled হয়ে 0 হয়েছে — ঠিক যেমন Standard Scaling-এ mean 0 হয়। Q1-এর নিচের values negative, Q3-এর উপরের values positive। কিন্তু output range 0 থেকে 1-এ fixed নয় — outlier থাকলে সেটা 1-এর বেশিও হতে পারে, কিন্তু বাকি normal values ঠিকঠাক scale হয়।

তিনটা Scaler-এর তুলনা

```
Property        |  Standard   |  MinMax     |  Robust
----------------|-------------|-------------|----------
Uses            |  Mean, Std  |  Min, Max   |  Median, IQR
Output range    |  Not fixed  |  0 to 1     |  Not fixed
Outlier effect  |  High       |  Very high  |  Low
Best for        |  Normal     |  No outlier |  Has outlier
```

কখন Robust Scaling ব্যবহার করবে?

Data-তে outlier থাকলে এবং সেই outlier remove বা clip করা সম্ভব না হলে Robust Scaling সবচেয়ে ভালো choice। এটা outlier-কে ignore করে না — outlier scaled data-তে থাকে, কিন্তু বাকি normal data-র scaling-কে নষ্ট করতে পারে না। তবে data normally distributed হলে এবং outlier না থাকলে Standard Scaling-ই বেশি উপযুক্ত।

Code

`marks_dataset.csv`-এ `maths_marks` column-এ Robust Scaling apply করা হয়েছে:

```
from sklearn.preprocessing import RobustScaler

rs = RobustScaler()
```

```
4
5# fit এবং transform
6rs.fit(df[['maths_marks']])
7df['maths_marks'] = rs.transform(df[['maths_marks']]).ravel()
8
9# scaling-এর পরে median = 0 হবে
10df['maths_marks'].describe()
```

RobustScaler-এ `fit()` করার সময় scaler training data-র median এবং IQR মনে রেখে দেয়। Transform-এর পরে distribution-এর shape বদলায় না — শুধু scale বদলায়, ঠিক আগের দুটো scaler-এর মতো।

Summary

Robust Scaling median এবং IQR ব্যবহার করে — এই দুটো outlier-এর প্রতি sensitive নয়। তাই data-তে outlier থাকলেও Robust Scaling বাকি normal values-কে সঠিকভাবে scale করতে পারে। তিনটা scaling পদ্ধতির মধ্যে outlier-এর presence বা absence দেখে সঠিকটা বেছে নেওয়াই হলো ভালো data scientist-এর কাজ। পরের পর্বে আমরা **ColumnTransformer** শিখবো — যেটা দিয়ে একসাথে অনেক transformation apply করা যায়।

ColumnTransformer

আগের পর্বগুলোতে আমরা scaling-এর তিনটা পদ্ধতি শিখেছি। কিন্তু একটু লক্ষ্য করো — `Feature_Engineering.ipynb`-এ আমরা প্রতিটা transformation আলাদা আলাদা করে apply করেছিলাম। Age-এর জন্য আলাদা imputer, Embarked-এর জন্য আলাদা imputer, Pclass-এর জন্য আলাদা encoder। এটা কাজ করে, কিন্তু code অনেক লম্বা এবং manage করা কঠিন হয়ে যায়। এই সমস্যার সমাধান হলো **ColumnTransformer**।

ColumnTransformer কী?

ColumnTransformer হলো scikit-learn-এর একটা tool যেটা দিয়ে বিভিন্ন column-এ বিভিন্ন transformation একসাথে এবং একটাই **step**-এ apply করা যায়। অনেকগুলো transformer-কে একটা single object-এ bundle করে রাখা যায়।

একটা analogy দিয়ে বুঝি। ধরো তুমি একটা factory-তে কাজ করছো। আগে পদ্ধতিতে — একজন worker Age ঠিক করে, তারপর আরেকজন Embarked ঠিক করে, তারপর আরেকজন Pclass encode করে — একটার পর একটা। ColumnTransformer-এ — সব worker একসাথে কাজ শুরু করে, প্রত্যেকে তার নিজের column নিয়ে কাজ করে, এবং সব শেষ হলে একসাথে output দেয়।

ColumnTransformer-এর Structure

```
from sklearn.compose import ColumnTransformer

transformer = ColumnTransformer(
    transformers=[
        ('name_1', transformer_1, ['column_1']),
        ('name_2', transformer_2, ['column_2', 'column_3']),
    ],
    remainder='passthrough'
)
```

প্রতিটা transformer তিনটা জিনিস নেয়:

```
Position  |  What it is
----------|-----------------------------------------
First     |  Name — যেকোনো string, identify করতে
Second    |  Transformer object — কী করবে
Third     |  Column list — কোন column-এ apply হবে
```

`remainder='passthrough'` মানে যে columns-এ কোনো transformation specify করা হয়নি, সেগুলো as-is রেখে দাও। `remainder='drop'` দিলে সেগুলো বাদ দিয়ে দেবে।

Titanic Dataset-এ ColumnTransformer

`Titanic_Feature_Engineering_via_columntransformer.ipynb`-এ দুটো আলাদা ColumnTransformer ব্যবহার করা হয়েছে — একটা imputation-এর জন্য, আরেকটা encoding এবং scaling-এর জন্য।

প্রথম ColumnTransformer — Imputation

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
import numpy as np

imputer_transformer = ColumnTransformer(
    transformers=[
        # Age column-এ mean imputation
        ('age', SimpleImputer(strategy='mean'), ['Age']),
```

```
        # Embarked column-এ most frequent imputation
        ('embarked', SimpleImputer(strategy='most_frequent'),
['Embarked']),

        # Cabin column-এ constant imputation + missing indicator
        ('cabin', SimpleImputer(
            strategy='constant',
            fill_value='Missing',
            add_indicator=True
        ), ['Cabin'])
    ],
    remainder='passthrough',
    verbose_feature_names_out=False
)
imputer_transformer.set_output(transform='pandas')

# fit এবং transform
imputer_transformer.fit(X_train)
X_train = imputer_transformer.transform(X_train)
X_test = imputer_transformer.transform(X_test)
```

তিনটা column-এ তিনটা আলাদা strategy — কিন্তু সব একটাই ColumnTransformer-এ।
`set_output(transform='pandas')` দিলে output numpy array-এর বদলে pandas DataFrame হয়।

দ্বিতীয় ColumnTransformer — Encoding এবং Scaling

```
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder
from sklearn.preprocessing import StandardScaler, MinMaxScaler

encoder_scaler = ColumnTransformer(
    transformers=[
        # Pclass — ordinal encoding
        ('pclass', OrdinalEncoder(
            categories=[['third', 'second', 'first']]
        ), ['Pclass']),

        # Embarked, Sex, Cabin_Deck — one hot encoding
```

```
        ('embarked_sex', OneHotEncoder(
            sparse_output=False, drop='first'
        ), ['Embarked', 'Sex', 'Cabin_Deck']),

        # Age — standard scaling
        ('age_scaler', StandardScaler(), ['Age']),

        # Fare, FamilySize — minmax scaling
        ('fare_scaler', MinMaxScaler(), ['Fare', 'FamilySize'])
    ],
    remainder='passthrough',
    verbose_feature_names_out=False
)
encoder_scaler.set_output(transform='pandas')

encoder_scaler.fit(X_train)
X_train = encoder_scaler.transform(X_train)
X_test = encoder_scaler.transform(X_test)
```

একটাই object-এ চারটা আলাদা transformation — OrdinalEncoding, OneHotEncoding, StandardScaling, এবং MinMaxScaling।

ColumnTransformer-এর সুবিধা

আগের approach-এ প্রতিটা transformer আলাদা করে fit এবং transform করতে হতো। ColumnTransformer-এ একটাই `fit()` এবং একটাই `transform()` — বাকি সব automatically হয়। এতে:

```
Benefit              |  Details
---------------------|-----------------------------------------
Less code            |  অনেক কম line-এ same কাজ
No mistake           |  Column miss হওয়ার সম্ভাবনা নেই
Pipeline ready       |  সহজে ML Pipeline-এ add করা যায়
Parallel execution   |  Transformations একসাথে apply হয়
```

fit() এবং transform() এর নিয়ম

ColumnTransformer-এও same নিয়ম — শুধু **training data**-তে **fit** করো, তারপর সেই fit করা transformer দিয়ে test data transform করো। এটা individual scaler-এর মতোই কাজ করে — শুধু অনেকগুলো transformer একসাথে bundle করা।

```
# সঠিক পদ্ধতি
transformer.fit(X_train)                # শুধু train দেখে শেখে
X_train = transformer.transform(X_train)
X_test = transformer.transform(X_test) # same transformer দিয়ে
```

Summary

ColumnTransformer বিভিন্ন column-এ বিভিন্ন transformation একসাথে apply করার সুযোগ দেয়। এতে code অনেক clean এবং manageable হয়। Titanic dataset-এ দুটো ColumnTransformer ব্যবহার করা হয়েছে — একটা imputation-এর জন্য, আরেকটা encoding এবং scaling-এর জন্য। এটা real-world machine learning pipeline-এর একটা গুরুত্বপূর্ণ অংশ।

Module Overview — Scaling

এই module-এ আমরা feature engineering-এর সবশেষ এবং গুরুত্বপূর্ণ একটা ধাপ শিখেছি — **Scaling**। শুরু থেকে শেষ পর্যন্ত একবার সব কিছু একসাথে দেখি।

আমরা কী কী শিখলাম?

Scaling কেন দরকার

Machine learning model বিভিন্ন feature-এর numeric value দেখে সিদ্ধান্ত নেয়। Feature-এর range আলাদা হলে বড় range-এর feature ছোট range-এর feature-কে dominate করে। Scaling সব feature-কে একটা common range-এ এনে এই সমস্যা সমাধান করে। Distance-based algorithms যেমন KNN, SVM, Linear Regression-এ scaling অবশ্যই দরকার।

তিনটা Scaling পদ্ধতি

```
Method    |  Formula              |  Output       |  Best for
----------|-----------------------|---------------|-----------------
Standard  |  (x - mean) / std     |  mean=0, std=1 |  Normal dist.
MinMax    |  (x - min)/(max-min)  |  0 to 1        |  No outliers
Robust    |  (x - median) / IQR   |  Not fixed     |  Has outliers
```

Standard Scaling data-কে z-score-এ convert করে। Mean 0 এবং std 1 হয়, কিন্তু distribution-এর shape বদলায় না। Outlier থাকলে mean এবং std distort হয় — তাই outlier-free normally distributed data-তে এটা সবচেয়ে ভালো।

MinMax Scaling data-কে 0 থেকে 1-এর মধ্যে নিয়ে আসে। Minimum value 0 এবং maximum value 1 হয়। Outlier থাকলে বাকি সব values একটা ছোট range-এ জমে যায় — তাই আগে outlier handle করতে হবে।

Robust Scaling median এবং IQR ব্যবহার করে। এই দুটো outlier-এর প্রতি sensitive নয় — তাই data-তে outlier থাকলেও বাকি values ঠিকমতো scale হয়।

সবচেয়ে গুরুত্বপূর্ণ নিয়ম

Scaler সবসময় **training data-**তে **fit** করতে হবে — কখনো test data-তে নয়। কারণ real world-এ নতুন data আসলে তাকে training distribution অনুযায়ী scale করতে হবে।

```
scaler.fit(X_train)                  # শুধু train data দেখে শেখে
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)    # same scaler দিয়ে transform
```

কোন Scaler কখন বেছে নেবে

```
Situation                          |  Best choice
-----------------------------------|----------------
Normal distribution, no outlier    |  Standard Scaling
No outlier, need 0-1 range         |  MinMax Scaling
Outliers present                   |  Robust Scaling
Neural network input               |  MinMax Scaling
```

ColumnTransformer

ColumnTransformer দিয়ে বিভিন্ন column-এ বিভিন্ন transformation একসাথে apply করা যায়। আলাদা আলাদা করে fit এবং transform করার বদলে একটাই object-এ সব bundle করা যায় — code হয় clean এবং manageable।

```
transformer = ColumnTransformer(
    transformers=[
        ('age_scaler',  StandardScaler(), ['Age']),
        ('fare_scaler', MinMaxScaler(),   ['Fare']),
    ],
    remainder='passthrough'
)
transformer.fit(X_train)
X_train = transformer.transform(X_train)
X_test  = transformer.transform(X_test)
```

Full Pipeline — Titanic Dataset

Titanic dataset-এ আমরা যা করেছি সেটা একবার দেখি:

```
Step  |  Task                     |  Method
------|---------------------------|-----------------------------------
1     |  Drop useless columns     |  PassengerId, Name, Ticket drop
2     |  Feature creation         |  FamilySize = SibSp + Parch + 1
3     |  Train-test split         |  80-20, random_state=42
```

```
6 4  |  Impute Age              |  Mean imputation
7 5  |  Impute Embarked         |  Most frequent imputation
8 6  |  Impute Cabin            |  Constant + missing indicator
9 7  |  Outlier — Age           |  Z-score, drop rows beyond 3
10 8  |  Outlier — Fare          |  IQR clipping
11 9  |  Encode Pclass           |  OrdinalEncoder
12 10  |  Encode Embarked/Sex     |  OneHotEncoder
13 11  |  Scale Age               |  StandardScaler
14 12  |  Scale Fare, FamilySize  |  MinMaxScaler
```

প্রতিটা step-এর একটা কারণ আছে। Age normally distributed — তাই StandardScaler। Fare-এ outlier আগেই clip করা হয়েছে — তাই MinMaxScaler। এই সিদ্ধান্তগুলো data দেখে নেওয়া হয়েছে, blindly নয়।

Summary

Scaling হলো feature engineering-এর শেষ ধাপ যেটা machine learning model-কে সঠিকভাবে শিখতে সাহায্য করে। তিনটা পদ্ধতি — Standard, MinMax, এবং Robust — প্রতিটার নিজস্ব use case আছে। ColumnTransformer এই সব transformation-কে একটা clean এবং reusable structure-এ রাখে। এই module-এর concepts গুলো যেকোনো real-world machine learning project-এ সরাসরি কাজে আসবে।